

Usuário:

Ola. Estou desenvolvendo um projeto de código de agente IA. No decorrer do processo e a IA que auxiliava nos perdemos e não conseguimos mais encontrar o fio da meada. Preciso que faça uma análise profunda da cópia do CHAT com a IA e o estágio atual que se encontra o projeto. Poderia através deste documento em formato json tentar levantar a estrutura atual do processo? Com base nesta estrutura, delinear os próximos passos para a conclusão do trabalho?

IA:

Estou analisando o arquivo Documentao-composta-do-Projeto-em-estrutura-JSON.txt para levantar a estrutura atual do seu projeto.

Alfa aqui. Vou analisar o documento completo para entender o estado atual do seu projeto. Vamos direto ao ponto.

Análise Completa do Projeto — Agente de IA Autônomo

Situação Atual do Projeto

Após analisar todo o histórico do chat, aqui está o **raio-X completo** do estado atual do seu projeto **Agente_Independente**.

O que já está construído e funcional (4 Pilares)

1. Sistema Bootstrap / Restore

Mecanismo de snapshot de projetos inteiros em JSON, com scripts PowerShell:

- ``bootstrap.ps1`` — exporta estrutura + conteúdo de arquivos para JSON
- ``restore_bootstrap.ps1`` — recria o projeto a partir do JSON
- ``Create_file_v3.ps1`` — cria arquivos a partir do clipboard com parsing de cabeçalho

- **Múltiplos bootstraps validados** (Agente_Independente, ProjetoCotacao, Interface_Adapta, etc.)

2. Command Protocol

Protocolo de comandos estruturados em JSON Schema para comunicação entre componentes.

3. Sistema de Patches

- ``Fix_File_v2.ps1`` — aplica patches no formato FIND/REPLACE com backup automático
- Validação de sintaxe antes da aplicação

4. Agente_Independente (Core)

Agente Python com módulos:

- ``src/analyser/`` — análise de código
- ``src/fixer/core.py`` — lógica central de correção
- ``src/fixer/agent.py`` — agente de correção
- ``src/validador/`` — validação (compatibility_validator.py)
- ``src/patches/executor.py`` — execução de patches
- ``src/utils/`` — utilitários
- ``guardrails.json`` — regras de segurança
- **24/24 testes passando** no Wizard of Oz com ProjetoCotacao

5. Banco de Dados PostgreSQL

- Banco agent_ia_automate com 5 tabelas: projects, project_versions, project_json_data, audit_log, validation_log

6. Módulo 2 — Expert Independente (VALIDADO)

- 8 arquivos Python criados e validados (100% — 0 erros críticos, 0 warnings)
- Estrutura: modulo2/0/ com expert_core.py, language_detector.py, patch_formatter.py, correctors/

7. Interface_Adapta (Camada RPA)

- Flask backend + interface web HTML/JS
- Integração com Adapta ONE para correção via Expert "Corretor de Código"

Ø=Áque está pendente / em andamento

As 6 Melhorias Críticas (identificadas na Sprint Planning)

#	Melhoria	Status
1	Teste de Compatibilidade — executar script pós-patch para garantir funcionamento	Código gerado (compatibility_validator.py, executor.py)
2	Geração Automática de Patches — sistema para gerar patches a partir de diffs	Não iniciado
3	Rollback Automático — comando para reverter patch automaticamente	Não iniciado
4	Histórico de Patches — dashboard/listagem de patches aplicados	Não iniciado
5	Patches Compostos — aplicar múltiplos patches em sequência	Não iniciado
6	Validação de Conteúdo — verificar se REPLACE não quebra lógica	Não iniciado

Módulo 3 — Agente Interno

- Planejado mas **não implementado**
- Seria o módulo que integra o Expert Independente com o Agente_Independente local

Documentação / README

- Mencionado mas não foi possível confirmar se foi finalizado

~~0=~~Próximos Passos — Roteiro para Conclusão

Considerando que você já tem uma base sólida, recomendo seguir nesta ordem:

Passo 1: Finalizar o Módulo 3 (Agente Interno)

Foi o último tópico discutido antes de perder o fio da meada. Este módulo conecta o Expert Independente (Módulo 2) com o Agente_Independente local. Sem ele, a integração entre os módulos não se completa.

O que precisa:

- modulo3/0/ com o core de integração
- Dispatcher que roteia requisições entre Módulo 1 (Skill), Módulo 2 (Expert) e Módulo 3 (Interno)
- Pipeline de normalização/validação

Passo 2: Teste de Compatibilidade (Melhoria #1)

O esqueleto foi gerado (`compatibility_validator.py`), mas precisa ser integrado ao fluxo principal do agente. Sem isso, patches são aplicados sem verificação pós-execução.

Passo 3: Geração Automática de Patches (Melhoria #2)

O protótipo manual (Wizard of Oz) mostrou que o agente consegue identificar erros e sugerir correções. Agora precisa gerar patches no formato FIND/REPLACE automaticamente a partir dos diffs.

Passo 4: Rollback Automático + Histórico (Melhorias #3 e #4)

Mecanismo para reverter patches problemáticos e manter um registro do que foi aplicado. Sem isso, o sistema não é seguro para uso em produção.

Passo 5: Patches Compostos + Validação de Conteúdo (Melhorias #5 e #6)

Funcionalidades avançadas que permitem aplicar múltiplas correções em sequência e validar se as alterações não introduzem novos bugs.

Riscos a Endereçar

1. **Dependência de Integração:** O fluxo completo depende do Adapta ONE + Interface_Adapta + Agente. Se qualquer ponto falhar, o feedback loop quebra.
2. **Segurança:** O sistema de patches modifica arquivos no disco. As guardrails existem mas precisam ser testadas contra casos de borda.
3. **Encoding:** Os bootstraps tiveram problemas recorrentes com encoding (UTF-8 vs Windows cp1252). Isso precisa ser padronizado.

Resumo

Você tem 70-80% do projeto construído. O core do agente, os scripts de infraestrutura, o banco de dados e o Módulo 2 estão prontos. Faltam:

- **Módulo 3** (integração final)
- **5 das 6 melhorias críticas** de segurança/resiliência
- **Testes de integração** entre todos os componentes

O maior risco é tentar implementar tudo de uma vez. Minha recomendação: foque em **fechar o Módulo 3 primeiro** para ter o sistema integrado funcionando, depois ataque as melhorias críticas em ordem de prioridade.

Se quiser, posso:

- **Gerar o código do Módulo 3** com base na arquitetura dos Módulos 1 e 2
- **Revisar e completar** o `compatibility_validator.py` existente
- **Criar um novo bootstrap** do estado atual do projeto para você ter um snapshot fresco
- **Montar um plano de sprint detalhado** com tarefas quebradas por dia

Qual caminho você quer seguir?

Usuário:

Brilhante análise, esta de parabéns. Vou copiar este trecho do CHAT para agregar ao relatório geral para fins de histórico do projeto. Uma observação em relação aos módulos: Veja se estou certo nesta afirmação: O módulo 1 seria usado para uma IA genérica, a qual receberia como SKILL toda informação necessária para ser incorporada ao Agente Autônomo atual, O módulo 3 seria prompt dedicado a criar um expert dedicado para atuar no Agente Autônomo atual. Parece que o módulo 2 seria um módulo auxiliar para automatizar todas as tarefas de validação de sintaxe de scripts, validação de fluxos, implementação de alterações de código via patches, e parece que até geração de arquivos txt de implementação de patches (não sei se autonomamente, ou com assistência da IA).

IA:

Estou preparando a resposta. A distinção entre os módulos de IA para agentes autônomos é um ponto crucial para o projeto.

Obrigado pelo reconhecimento. Vou validar seu entendimento ponto a ponto com base no que o documento mostra.

Sua Afirmação vs. Realidade dos Módulos

Módulo 1 — Skill Adapta ONE

> Sua descrição: "Usado para uma IA genérica, que receberia como SKILL toda informação necessária para ser incorporada ao Agente Autônomo atual"

Correto no conceito geral. O Módulo 1 é uma **Skill reutilizável na plataforma Adapta ONE**. Ela encapsula o conhecimento e o comportamento do corretor para que qualquer IA genérica no Adapta ONE possa atuar como o agente de correção de código. É a camada de **interface com a plataforma de IA**.

Detalhe importante: Ela não contém *todo* o conhecimento do projeto — ela é o ponto de entrada que invoca as capacidades dos outros módulos. Pense como a "capa de apresentação" do sistema.

Módulo 2 — Expert Independente

> Sua descrição: "Módulo auxiliar para automatizar tarefas de validação de sintaxe, validação de fluxos, implementação de alterações via patches e geração de arquivos de implementação"

Substancialmente correto. Módulo 2 é o **Expert Independente**, e sim, ele cobre:

Capacidade	Como funciona
Validação de sintaxe	correctors/ + skills #Validar Sintaxe Python / #Validar Sintaxe PowerShell
Detecção de linguagem	language_detector.py
Formatação de patches	patch_formatter.py (gera FIND/REPLACE estruturado)
Geração de correções	expert_core.py + corretores especializados

Sobre sua dúvida (autônomo vs. assistido): O Módulo 2 foi projetado para operar **de forma autônoma na geração dos patches**, mas a validação final e a aprovação para aplicar as alterações passam por um crivo — o `compatibility_validator.py` valida o patch antes da aplicação. Então: **geração autônoma, aplicação com validação.**

Módulo 3 — Agente Interno

> Sua descrição: "Prompt dedicado a criar um expert dedicado para atuar no Agente Autônomo atual"

Não é bem assim. Essa foi a parte que mais se perdeu no histórico. O Módulo 3 não é um "prompt" — ele foi concebido como o **Agente Interno**, um módulo de **software** que faz a integração entre os outros módulos e o agente local.

A arquitetura original era:

...

Módulo 1 (Skill Adapta ONE)

!“

Módulo 3 (Agente Interno) !•% Roteador + Dispatcher

!“

Módulo 2 (Expert Independente)

!“

Agente_Independente (core local)

...

O Módulo 3 seria o **cérebro orquestrador** que:

- Recebe requisições do Módulo 1
- Roteia para o Módulo 2 ou diretamente para o Agente_Independente
- Gerencia o pipeline interno de validação/normalização
- Decide qual módulo usar dependendo do tipo de correção

Ele não foi implementado. Ficou como planejamento — e é o principal gargalo para fechar a integração completa do sistema.

Correção Crítica na Sua Afirmação

Você inverteu parcialmente os papéis:

...

Você disse: Realidade:

Módulo 1 = Skill genérica Skill Adapta ONE

Módulo 3 = Expert dedicado É o Agente Interno (orquestrador)

Módulo 2 = Auxiliar validação Expert Independente

...

O Módulo 3 não é um prompt — é código Python que faria o papel de **roteador/dispatcher** entre os módulos. O expert dedicado que você descreveu já existe: é o **Módulo 2**.

Resumo

Módulo	O que é	Status
Módulo 1	Skill Adapta ONE — interface da IA genérica com o sistema	Planejado
Módulo 2	Expert Independente — correção, validação, patches	Criado e validado (8 arquivos)
Módulo 3	Agente Interno — orquestrador/roteador entre módulos	Não implementado

Se quiser, posso ajudar com o **design detalhado do Módulo 3** para você incluir no relatório geral do projeto — incluindo a interface entre os módulos e o fluxo de decisão do roteador.

Usuário:

Eu acho que então precisaríamos criar um quarto módulo (módulo 4: expert Adapta ONE),. Você poderia tentar projetar a estrutura deste módulo, e como ele se conecta ao restante do Sistema? Também rebatizar os Módulos 2 e 3: Módulo 2 : Módulo de Debug autonomo . Módulo 3: Orquestrador autonomo. Módulo 4: Expert: Agente de Projeto e Debug de Codigos . O que voce acha?

IA:

Estou analisando essa proposta de reestruturação. A ideia de um módulo "Expert" para automação de projetos e depuração de código parece interessante.

Brilhante sua observação. Você identificou um gap real na arquitetura original.

A diferença entre uma **Skill** (instruções reutilizáveis para qualquer IA) e um **Expert** (um agente de IA dedicado com contexto persistente) é enorme — e de fato faltava essa camada no design.

Vou projetar a reestruturação completa e o novo Módulo 4.

Análise do seu Proposta

Sua intuição está correta em dois pontos fundamentais:

- 1. **Skill "Expert**: O Módulo 1 como Skill permite que *qualquer* IA genérica no Adapta ONE use as instruções do corretor, mas é leve e sem estado. O Expert (Módulo 4) é um **agente de IA dedicado** com memória, contexto de projeto e autonomia.
- 2. **Missing middle**: Entre a Skill genérica e o código local do Agente_Independente, faltava uma camada de **inteligência com consciência de projeto**. O Expert preenche esse vazio.

Nova Arquitetura de Módulos

Para visualizar:

Validação da Proposta de Renomeação

Aprovado. Sua nomenclatura é mais precisa que a original:

Nome Antigo	Novo Nome	Status
Módulo 1 — Skill Adapta ONE	Módulo 1 — Skill Adapta ONE	Mantido
Módulo 2 — Expert Independente	Módulo 2 — Debug Autônomo	Muito mais descritivo
Módulo 3 — Agente Interno	Módulo 3 — Orquestrador Autônomo	Reflete o papel real
inexistente	Módulo 4 — Expert: Agente de Projeto e Debug de Códigos	Novo

Agora vamos ao projeto do Módulo 4.

Design do Módulo 4 — Expert: Agente de Projeto e Debug de Códigos

Papel no Ecossistema

O Módulo 4 é o **Expert dedicado dentro do Adapta ONE**. Enquanto o Módulo 1 é uma Skill genérica que qualquer IA pode carregar, o Módulo 4 é um **agente de IA especializado** — ele tem identidade, contexto e autonomia.

Pense assim: a Skill (Módulo 1) é como um **manual de instruções** que qualquer pessoa pode ler. O Expert (Módulo 4) é o **técnico especialista** que já leu o manual, conhece o projeto, e age.

Estrutura Interna Proposta

```
```text
```

```
modulo4/
```

```
% %0%
```

```
% % %expert_core.py # Núcleo do Expert — identidade, inicialização
```

```
% % %project_context.py # Gerencia o contexto do projeto atual
```

```
% % %project_memory.py # Memória persistente — decisões, patches, erros
```

```
% % %project_mapper.py # Mapeia estrutura completa do projeto (bootstrap-aware)
```

```
% % %session_manager.py # Gerencia sessões de depuração
```

```
% % %orchestrator_client.py # Interface de comunicação com Módulo 3
```

```
% % %skill_bridge.py # Interface de comunicação com Módulo 1
```

```
% % %user_interface.py # Camada de interação com o usuário no Adapta ONE
```

```
% % %decision_engine.py # Lógica de decisão — quando agir, o que perguntar
```

```
% % %report_generator.py # Gera relatórios de diagnóstico e progresso
```

```
% % %prompt_templates/
```

```
% % %system_prompt.md # Prompt de sistema do Expert
```

```
% % %analyze_prompt.md # Template para análise de código
```

...

## Como o Módulo 4 se Conecta ao Restante do Sistema

# Fluxo de Alta — Caminho Principal

```
```text

Usuário

%

%1/4

Módulo 4 — Expert: Agente de Projeto e Debug !•% 7 Entrada principal

%

% % project_context.py (carrega bootstrap do projeto)

% % project_mapper.py (mapeia estrutura)

% % decision_engine.py (define estratégia)

%

% % Módulo 3 — Orquestrador Autônomo

% %

% % % Módulo 2 — Debug Autônomo

% % %

% % % % Validação de sintaxe

% % % % Geração de patches

% % % % Corretores especializados

% % % % Testes de compatibilidade

% % %

% % % Agente_Independente (core local)

%

% % report_generator.py (responde ao usuário)

```
```

## Fluxo Alternativo — Uso Rápido

```text

Usuário

%

%¼

Módulo 1 — Skill Adapta ONE (IA genérica carrega a skill)

%

% % Módulo 3 — Orquestrador !•% % Mesmo pipeline, sem o Expert

%

% % Usuário recebe resposta sem contexto avançado

```

## Tabela de Comparação: Módulo 1 vs. Módulo 4

Aspecto	Módulo 1 — Skill	Módulo 4 — Expert
Natureza	Instruções reutilizáveis	Agente de IA dedicado
Contexto	Nenhum — carrega instruções apenas	Persistente — sabe o projeto inteiro
Memória	Efêmera (só a conversa atual)	Persistente (histórico de patches, decisões)
Autonomia	Baixa — executa o que a IA hospedeira decide	Alta — toma decisões, age, reporta
Ativação	Toda IA no Adapta ONE pode usar	Criado como Expert específico
Complexidade	Leve (1 arquivo de instrução)	Pesado (múltiplos módulos Python)



- Analisa o pedido do usuário como um desenvolvedor experiente
- Consulta o contexto do projeto e o histórico



- Decide a melhor estratégia de correção
- Aciona o Orquestrador com instruções precisas
- Interpreta os resultados e reporta em linguagem natural
- Aprende com cada ciclo para melhorar as próximas iterações

## Resumindo

- Sua proposta de renomeação foi validada — os novos nomes são mais precisos e descritivos
- O Módulo 4 preenche o gap mais crítico: a **falta de um agente dedicado com contexto persistente** entre a Skill genérica (Módulo 1) e o código local
- A arquitetura agora tem **2 modos de operação**: via Skill (leve/rápida) e via Expert (completa/profunda)
- O Módulo 4 reutiliza toda a infraestrutura já construída — bootstraps, PostgreSQL, patches, scripts — não começa do zero
- A conexão é clara: Expert !' Orquestrador !' Debug Autônomo !' Core Local

Se quiser, posso:

- **Gerar o código base do Módulo 4** — criar a estrutura de pastas e os arquivos Python esqueleto de todos os componentes
- **Escrever o system\_prompt.md** do Expert — a identidade, regras e tom do Agente de Projeto e Debug
- **Detalhar a interface** entre Módulo 4 e Módulo 3 — o protocolo de comunicação (formato dos comandos, respostas, erros)
- **Atualizar os bootstraps** dos Módulos 2 e 3 com os novos nomes e salvar o estado atual do projeto

Qual caminho você quer seguir primeiro?